

WEEK 3

SAREE BOUTIQUE

Seller and Inventory Management System

Database Activity Report using Oracle SQL

Module	Seller and Inventory Management
Database	Oracle SQL
Main Tables	SELLER, INVENTORY
Key Concepts	Primary Key, Foreign Key, UNIQUE, NOT NULL, CHECK
Reports	Seller-wise Product, Stock Available, Inventory Analysis

Scenario

Saree Boutique – Seller and Inventory Management System is a database system designed to maintain seller details and track the stock of saree products. The system stores seller contact information, product inventory, stock quantity and stock status. Relationships and constraints are added to maintain data accuracy and consistency.

The activity demonstrates table creation, relationships, primary and foreign keys, data integrity constraints, insertion of records, and SQL-based inventory reports.

1. Create Seller and Inventory Tables

Definition: Tables are database structures used to store related information. The SELLER table stores seller information such as seller ID, seller name, email, phone and address. The INVENTORY table stores inventory ID, product ID, seller ID, stock quantity, stock status and last update date.

Seller table creation

```
SQL> CREATE TABLE seller(seller_id varchar(7),seller_name varchar(25),e
mail varchar(30),phone number(10),address varchar(45));

Table created.
```

Inventory table creation

```
SQL> CREATE TABLE inventory(inventory_id varchar(7),product_id varchar(
7),seller_id varchar(7),stock_quantity number,stock_status varchar(11),
last_update DATE);

Table created.
```

2. Establish Relationships

Seller and Saree Product Relationship: A seller can supply one or more products. The seller is identified by SELLER_ID, while products are identified by PRODUCT_ID.

Product and Inventory Relationship: The INVENTORY table stores PRODUCT_ID as a reference to the PRODUCT table. This connects each inventory record with its corresponding product.

3. Primary Key Added for Both Tables

Definition: A primary key uniquely identifies each row in a table. It cannot contain duplicate values and is used to uniquely identify records.

```
SQL> ALTER TABLE seller ADD CONSTRAINT pk_seller PRIMARY KEY(seller_id)
;

Table altered.

SQL> ALTER TABLE inventory ADD CONSTRAINT pk_inv PRIMARY KEY(inventory_
id);

Table altered.
```

4. Foreign Key for INVENTORY Table

Definition: A foreign key creates a relationship between tables by referencing a primary key in another table. Here, INVENTORY.PRODUCT_ID references PRODUCT.PRODUCT_ID, and INVENTORY.SELLER_ID references SELLER.SELLER_ID.

```
SQL> ALTER TABLE inventory ADD CONSTRAINT fk_prod FOREIGN KEY(product_i
d) REFERENCES product(product_id);

Table altered.

SQL> ALTER TABLE inventory ADD CONSTRAINT fk_seller FOREIGN KEY(seller_
id) REFERENCES seller(seller_id);

Table altered.
```

5. Constraint to Make Email ID Unique

Definition: A UNIQUE constraint prevents duplicate values in a column. It ensures that the same seller email address cannot be entered more than once.

```
SQL> ALTER TABLE seller ADD CONSTRAINT un_mail UNIQUE(email);  
  
Table altered.
```

6. Make Email NOT NULL

Definition: A NOT NULL constraint ensures that a column must contain a value. For the seller table, every seller must have an email address.

```
SQL> ALTER TABLE seller MODIFY email NOT NULL;  
  
Table altered.
```

7. Check Stock Quantity is Positive

Definition: A CHECK constraint restricts values according to a condition. The stock quantity constraint allows only values greater than or equal to zero, preventing invalid negative stock quantities.

```
SQL> ALTER TABLE inventory ADD CONSTRAINT ck_quan CHECK(stock_quantity  
>=0);  
  
Table altered.
```

8. Insert Values into SELLER Table

Definition: INSERT is used to add new records to a table. The screenshot shows a seller record being inserted using substitution variables for seller ID, name, email, phone and address.

```
SQL> INSERT INTO seller VALUES('&sellerllid','&sellername','&email','&phone','&address');
Enter value for sellerllid: S00001
Enter value for sellername: vani enterprises
Enter value for email: mdvanienter@gmail.com
Enter value for phone: 8765432109
Enter value for address: anna nagar, chennai
old 1: INSERT INTO seller VALUES('&sellerllid','&sellername','&email','&phone','&address')
new 1: INSERT INTO seller VALUES('S00001','vani enterprises','mdvanienter@gmail.com','8765432109','anna nagar, chennai')

1 row created.
```

9. Insert / View Inventory Values

Definition: Inventory records associate products with sellers and store their current stock quantity and status. The supplied output shows the inventory records and their stock status.

```
SQL> SELECT s.seller_name, COUNT(i.product_id) AS number_of_products, SUM(i.stock_quantity) AS Available_stock FROM seller s JOIN inventory i ON s.seller_id = i.seller_id WHERE i.stock_status = 'Available' GROUP BY s.seller_name;

SELLER_NAME          NUMBER_OF_PRODUCTS  AVAILABLE_STOCK
-----
ABC Enterprises              1                3
DEF Enterprises              1                3
vani enterprises            1                4

SQL> SELECT p.product_name, i.stock_quantity, i.stock_status FROM product p JOIN inventory i ON p.product_id = i.product_id;

PRODUCT_NAME
-----
STOCK_QUANTITY STOCK_STATU
-----
Laptop          4 Available
Laptop          3 Available
Mobile Phone    5 Unavailable

PRODUCT_NAME
-----
STOCK_QUANTITY STOCK_STATU
-----
Keyboard        3 Available
```

Report 1 – Seller-wise Product Report

Definition: A seller-wise product report groups inventory records by seller. It shows the number of available products and the total available stock for each seller.

```
SQL> SELECT s.seller_name, COUNT(i.product_id) AS number_of_products, SUM(i.
stock_quantity) AS Available_stock FROM seller s JOIN inventory i ON s.selle
r_id = i.seller_id WHERE i.stock_status = 'Available' GROUP BY s.seller_name
;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
ABC Enterprises	1	3
DEF Enterprises	1	3
vani enterprises	1	4

Report 2 – Stock Available Report

Definition: A stock available report identifies products that are currently available and summarizes the number of available inventory records.

```
SQL> SELECT COUNT(*) AS total_Available_products FROM inventory WHERE stock_
status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
3

Report 3 – Inventory Analysis

Definition: Inventory analysis uses aggregate SQL functions such as AVG and MAX to understand stock levels and identify the highest-stocked product.

Highest Stocked Saree Product

Finding: The supplied dataset shows **Mobile Phone** with the highest stock quantity of **5**. The screenshot is reproduced as provided; the product name in the source image is not a saree name.

```
SQL> SELECT p.product_name,i.stock_quantity FROM product p JOIN inventory i
ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX (stock_q
uantity) FROM inventory);
```

```
PRODUCT_NAME
```

```
-----
```

```
STOCK_QUANTITY
```

```
-----
```

```
Mobile Phone
```

```
5
```

Average Inventory Quantity

Finding: The average inventory quantity in the supplied dataset is **3.75**, calculated using AVG(stock_quantity).

```
SQL> SELECT AVG(stock_quantity) AS average_inventory FROM inventory;
```

```
AVERAGE_INVENTORY
```

```
-----
```

```
3.75
```

Additional Inventory Analysis Outputs

The supplied screenshots also contain the following supporting analysis results.

Out-of-stock Products

The query output shows **0** out-of-stock products for the supplied data.

```
SQL> SELECT COUNT(*) AS outofstock_products FROM inventory WHERE stock_status = 'out of stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

Inventory Product Status

```
SQL> SELECT s.seller_name, COUNT(i.product_id) AS number_of_products, SUM(i.stock_quantity) AS Available_stock FROM seller s JOIN inventory i ON s.seller_id = i.seller_id WHERE i.stock_status = 'Available' GROUP BY s.seller_name;

SELLER_NAME          NUMBER_OF_PRODUCTS  AVAILABLE_STOCK
-----
ABC Enterprises              1                3
DEF Enterprises              1                3
vani enterprises            1                4

SQL> SELECT p.product_name, i.stock_quantity, i.stock_status FROM product p JOIN inventory i ON p.product_id = i.product_id;

PRODUCT_NAME
-----
Laptop
4 Available
Laptop
3 Available
Mobile Phone
5 Unavailable

PRODUCT_NAME
-----
Keyboard
3 Available
```

Conclusion

The Week 3 activity demonstrates how an Oracle SQL database can be used to manage seller and inventory information. Primary keys provide unique identification, foreign keys establish relationships, and UNIQUE, NOT NULL and CHECK constraints improve data integrity. Aggregate queries provide useful inventory summaries such as available stock, highest stock and average quantity.